BoxBox — Estrategia de neumáticos en Fórmula 1

Trabajo Práctico Integral — Entrega N.º 2: prototipo y evaluación
Inteligencia Artificial Avanzada — UTN FRBA — 2do Cuatrimestre 2026

Apellido y Nombres	E-Mail	Aporte
Sánchez, Tomás Agustín	tosanchez@frba.utn.edu.ar	100%

Fecha de Presentación: 22 de octubre de 2026

Repositorio: https://github.com/tomasanchez/boxbox

> *Versión reducida. El informe completo —con el detalle de cada medición, los casos resueltos uno por uno y los errores propios documentados— está en* `informe.pdf` *, mismo repositorio.*

Resumen

En una carrera de Fórmula 1 el equipo tiene que decidir cuándo parar en boxes y qué neumático poner. Este trabajo construye un sistema que **arma ese plan**: cuántas paradas, en qué vueltas y con qué compuesto. No predice quién gana.

La pregunta que organiza el informe es si funciona, y la respuesta es que como constructor de planes sí, y como pronosticador de carreras concretas todavía no. Se probó contra cuatro carreras reales de 2026:

Prueba	Veredicto
Monza, fecha 13	falló: recomendó una parada, el ganador hizo dos — 18 s de error
Madrid, fecha 14	acertó la forma del plan ganador, falló el desgaste por un factor de tres
Compuesto de salida, grilla de 22	falló: pone medio 0 de 22 y la realidad lo pone entre 55% y 85%
Bakú, fecha 15	1 de 4 criterios, escritos y commiteados **antes** de la carrera

Tres de esas cuatro las decidió **una neutralización**, y ahí está la diferencia entre dos problemas: una bandera cambia la estrategia **en tiempo real**, y lo que este trabajo entrega es un plan **pre-carrera** que sólo puede usar la **probabilidad** de que aparezca.

Lo que sí quedó demostrado. En Madrid, volviendo a correr la búsqueda con el desgaste medido *en la carrera* y sin tocar nada más, propone `M17-H40` contra el `M14-H43` del ganador: tres vueltas. **El algoritmo no estaba roto; los insumos sí.**

Sobre el método: le gana **casi once segundos** de carrera a cualquier regla fija enunciable, genera **cero planes ilegales**, y una medición de esta entrega muestra que **entrena de más**.

---

1. Introducción

Cada vuelta, el ingeniero de estrategia responde una pregunta simple de enunciar y difícil de contestar: ¿entro a boxes ahora o sigo? Si entra, pierde **22,6 segundos** en la mediana —medido sobre cinco temporadas, y mucho menos si hay bandera—. Si no entra, pierde tiempo con la goma gastada. Y si sale un Safety Car justo después, la decisión que parecía buena se vuelve mala.

Objetivo: construir un sistema que, para una carrera dada, calcule el plan de neumáticos más conveniente y explique por qué.

El sistema entrega, para cada piloto: **un plan completo** (en qué vueltas parar y qué compuesto calzar), **cómo se repartió la población de la búsqueda** entre cantidades de parada, **una ventana de vueltas** para la parada expresada en los puestos que cuesta moverla, y **la distribución de puestos de llegada** sobre las carreras sorteadas.

Qué queda afuera: la gestión de energía. En 2026 la mitad de la potencia es eléctrica, pero la Fórmula 1 no publica esos datos y los canales de telemetría no los traen.

---

2. Materiales Disponibles

La fuente es **FastF1**, que expone los datos oficiales de cronometraje de la FIA. No hay fuentes privadas ni datos propios.

temporada	carreras	pilotos	vueltas
2022	22	22	23.577
2023	22	22	24.420
2024	24	24	26.604
2025	24	21	26.689
2026	14	23	16.255
total	**106**		**117.545**

Las temporadas viejas entran aunque el reglamento haya cambiado: se midió que agregarlas baja el error del modelo de degradación un 56%.

Preparación de los datos

De las 33 columnas crudas se derivan 20 más. Las tres transformaciones que importan:

1. **Corrección por avance de carrera.** Un auto con el tanque lleno es más lento que con el tanque vacío, y eso se confunde con desgaste. Se descuenta un coeficiente fijo de 0,035 s/vuelta, calibrado sobre 1.627 carreras-piloto.
2. **Degradación relativa a la propia tanda.** El objetivo no es el tiempo de vuelta absoluto —cada circuito aparece una vez por temporada y un modelo de tiempo absoluto tendría que predecir una pista que nunca vio— sino los **segundos perdidos contra la referencia de la misma tanda**. Eso sí transfiere.
3. `TyreLife` **no es «vueltas de esta tanda».** Un equipo puede calzar un juego usado en cualquier parada. Medido: las dos columnas coinciden en el **80,7%** de las vueltas y difieren en el resto, con mediana de 2 vueltas y máximo de 25.

Limpieza: qué se descarta y por qué

paso	vueltas	%	pierde	qué exige
crudas de FastF1	117.545	100,0%	—	todo lo que devuelve la API
con tiempo de vuelta	115.715	98,4%	1.830	descarta abandonos y vueltas sin cronometrar
representativas	101.273	86,2%	14.442	no es entrada ni salida de boxes, no fue borrada por los comisarios, y pasa el control `IsAccurate`
en seco	95.782	81,5%	5.491	fuera intermedio y lluvia, que el simulador no representa

No hay un paso «en verde» aparte: de las 8.571 vueltas corridas bajo alguna bandera, **todas fallan el control** `IsAccurate`, así que el filtro de representativas ya se las llevó. Filtrar por bandera después no saca ninguna fila más — es redundante, no incorrecto.

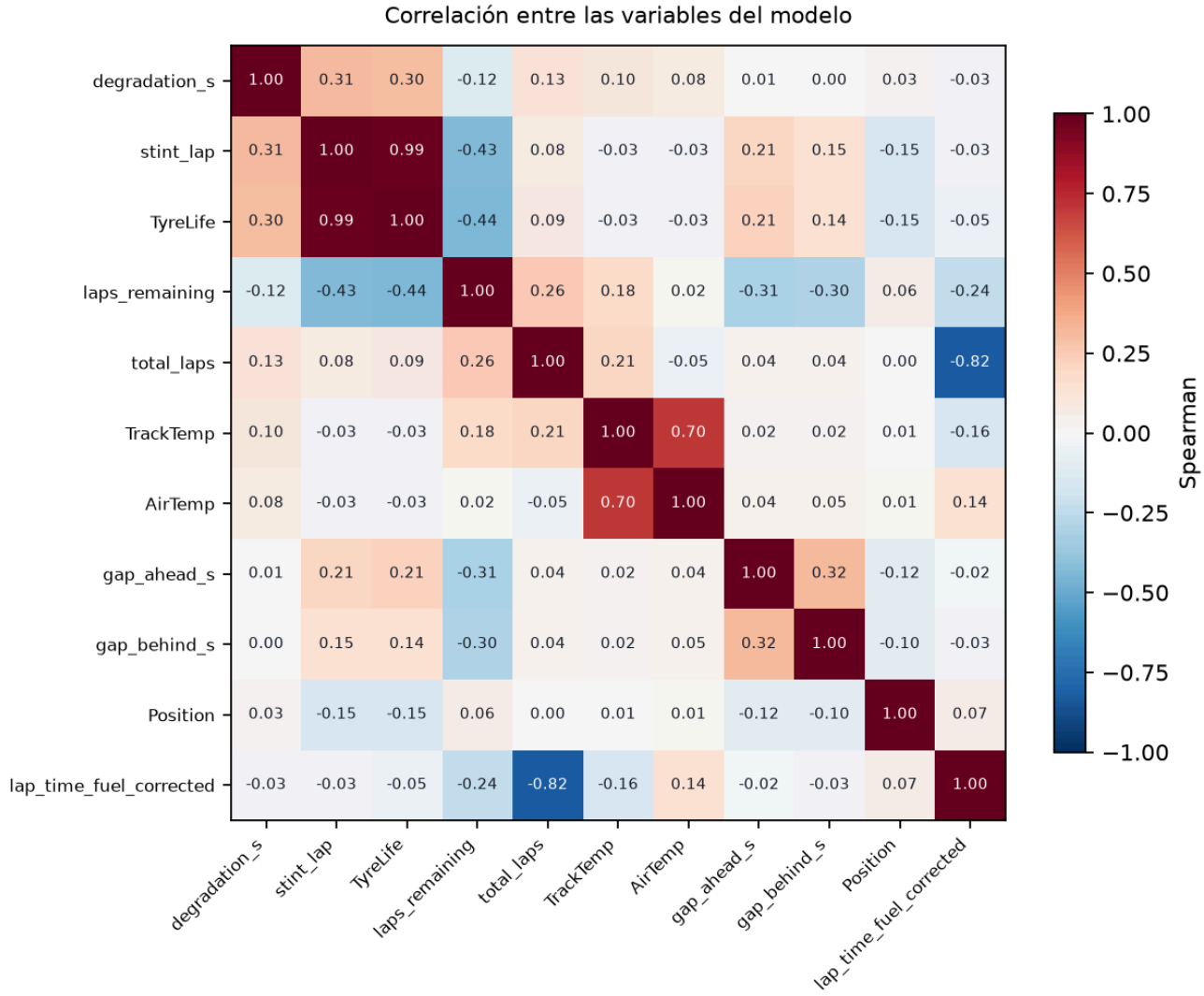
Qué se intenta predecir

`degradation_s`: segundos perdidos contra la referencia de la propia tanda.

compuesto	vueltas	mediana	desvío
duro	46.664	0,391	1,61
medio	36.717	0,335	1,857
blando	12.401	0,419	3,086

El desvío es mayor que la mediana en los tres: la incertidumbre de una vuelta es más grande que la señal que se intenta predecir. Es lo que justifica simular en vez de estimar un valor.

Las variables entre sí



Tres lecturas:

1. `stint_lap` y `TyreLife` **correlacionan 0,99**, pero no son la misma variable: coinciden al 100% en las vueltas de juego nuevo (80,7%) y sólo al 8,2% en las de juego usado. Lo que las separa es `FreshTyre`, que es otra columna del modelo — la redundancia es **de tres columnas, no de dos**.
2. `total_laps` **y el tiempo de vuelta correlacionan −0,82**, que es una identidad disfrazada: las carreras largas son las de vuelta corta. Las dos temperaturas van a 0,70.
3. **El objetivo no correlaciona con nada por encima de 0,31**, y el resto queda en 0,13 o menos. Ésa es la cifra que explica el resto del trabajo: un regresor sobre estas columnas no puede ir mucho más allá de la mediana. No invalida el enfoque, lo redirige — si el valor de una vuelta no se puede predecir, queda describir bien su *distribución* y simular.

Método de selección: entrenamiento, validación y prueba

La partición es **por carrera y cronológica**: las últimas **tres de las doce** fechas de 2026 quedan retenidas y nunca se ven durante el entrenamiento ni el ajuste. Tres de doce es el **25%** que exige la consigna, y la unidad es la carrera porque partir por vuelta pondría vueltas de la

misma tanda de los dos lados y filtraría información.

Retener las carreras **más recientes** en vez de tres al azar refleja la tarea real: el sistema tiene que pronosticar fechas que todavía no ocurrieron. La validación interna es `GroupKFold` de cuatro pliegues, agrupando por carrera, dentro del conjunto de entrenamiento.

---

3. Solución Propuesta

Tres piezas, y sólo dos están construidas.

1. Simulador Monte Carlo (el entorno). Simula una carrera vuelta a vuelta: cuánto se degrada cada goma, cuánto cuesta parar y si sale o no una neutralización. Se corre 1.200 veces por evaluación. No es una técnica de IA: es el banco de pruebas de las que sí lo son.

2. Algoritmo Genético (la técnica principal — Unidad 3). Lo que se entrega.

3. Agente de Aprendizaje por Refuerzo (Unidad 5) — ⚠️ NO CONSTRUIDO. No hay una sola línea de código de aprendizaje por refuerzo en el repositorio. Se menciona porque es la comparación que el trabajo propone y porque la Unidad 5 se dicta para la Entrega 3.

Cómo funciona el algoritmo genético

El individuo. Un plan es una secuencia de paradas de largo variable, de cero a cuatro. Cada parada lleva la vuelta en la que entra y el compuesto que calza. El genoma *es* el plan.

La población inicial se siembra con los planes obvios —repartir la distancia en partes iguales para una, dos, tres y cuatro paradas— y se completa al azar. Sembrar importa: 118 planes de dos paradas al azar dieron como mejor 93,3 s contra 91,0 del reparto parejo hecho a mano.

Selección: elitismo explícito. Cada generación se ordena por aptitud y la cuarta parte superior pasa intacta. Los hijos se arman tomando un padre de esa élite y el otro de la población entera.

Cruza en un punto, cortando por vuelta y no por índice, para que el hijo tenga una primera mitad de carrera coherente y una segunda mitad coherente.

La regla de mutación, que se aplica al 60% de los hijos:

Sorteo	Operación	Condición
< 0,15	**agrega** una parada en una vuelta al azar, compuesto al azar	si tiene menos de 4
0,15 – 0,30	**quita** una parada al azar	si tiene alguna
0,30 – 0,65	**mueve** una parada entre −6 y +6 vueltas	si tiene alguna
> 0,65	**cambia el compuesto** de una parada	si tiene alguna

Reparación, después de cada operador. Es la pieza que no es estándar. Todo hijo pasa por un reparador que impone paradas ordenadas y separadas al menos 6 vueltas; ninguna tanda más larga que la vida medida de su compuesto (41 el duro, 31 el medio, 25 el blando); y los **dos compuestos secos** que exige el artículo B6.3.8.

El reglamento no actúa como filtro, actúa como reparador. Un plan ilegal no se descarta: se le cambia el compuesto de la última parada por uno que el auto no haya usado. Descartarlo sesgaría la búsqueda hacia lo que resultó legal por casualidad. Por eso la cantidad de planes ilegales generados es **cero** y no hace falta penalizarlos en la aptitud.

El ciclo son 25 generaciones de 40 individuos: 1.040 evaluaciones, cada una sobre 1.200 carreras sorteadas.

Topología



Lo verde es el entorno; lo naranja, la técnica de la Unidad 3.



Herramientas

Las que se usan: Python 3.13, `fastf1`, `pandas`, `numpy`, `DEAP`, `LightGBM`, `scikit-learn`, `matplotlib`; React 19, Vite y MUI en la interfaz; cuadernos Jupyter vía `jupytext`.

Declarada y sin usar: `gymnasium`, puesta para el agente de la Entrega 3. No la importa ningún módulo, y se la nombra en vez de esconderla porque una dependencia que nadie usa es una promesa, no una herramienta.

Sobre DEAP, que la cátedra recomienda. Las dos implementaciones existen y funcionan, pero el valor por omisión es el motor propio y **todas las cifras publicadas salen de él** (`"engine": "builtin"` en la metadata del export). DEAP se usa para **contrastar**: sobre 75 problemas los dos motores dan el mismo plan en el **81,3%** de los casos, ninguno le gana al otro de forma sistemática, y el propio es un 20% más rápido. Habiendo medido eso, producir con el motor propio es una decisión justificada.

---

4. Desarrollo

Las corridas: qué parámetros se probaron

Cinco semillas por combinación, todas juzgadas sobre las mismas 20.000 carreras retenidas.

#	Sorteos	Población	Generaciones	Tiempo medio	Dispersión	
1	200	40	25	94,36 s	0,51	
2	400	40	25	94,05 s	0,40	
3	800	40	25	**93,99 s**	**0,21**	mejor
4	1.200	40	25	94,05 s	0,57	← la que corre
5	1.200	20	25	94,23 s	1,17	
6	1.200	80	25	94,32 s	1,17	
7	1.200	40	10	94,03 s	0,59	
8	1.200	40	50	94,32 s	1,17	

Tres lecturas, ninguna la esperada: **más sorteos dejan de pagar antes de los 1.200** que usa producción; **población y generaciones casi no mueven la aguja, y de más empeora**, porque el espacio es chico y el reparador descarta lo ilegal antes de evaluar; y **diez generaciones empatan con veinticinco**.

> **La advertencia que la tabla se hace a sí misma.** La dispersión entre semillas va de 0,21 a 1,17 s y las diferencias entre configuraciones son de 0,05 a 0,33. Con cinco semillas esto **no puede ordenar las filas con confianza**: distingue la de 200 del resto y poco más.

Contra qué se compara

Regla	Ventaja del algoritmo
«siempre 1 parada, duro»	**+10,80 s**
«siempre 2 paradas, duro»	+0,07
«siempre 3 paradas, duro»	**+10,91 s**
el oráculo, que ya sabe la respuesta	+0,07

El algoritmo le gana a cualquier regla enunciable, y lo que gana es exactamente el valor de acertar la cantidad de paradas — que son 12,5 segundos. La vuelta exacta de la parada vale 0,30 segundos.

Problemas e inconvenientes, y cómo se resolvieron

Los problemas no fueron de programación: fueron **mediciones que daban un número confiado y plausible, y estaban mal**. Ninguno se cayó solo; cada uno se descubrió porque algo aguas abajo salió absurdo. Los principales:

Problema	Cómo apareció	Resolución
El coeficiente de combustible sacaba el 63% del efecto	el desgaste medido daba casi cero	se recalibró sobre 1.627 carreras-piloto
El ritmo extrapolado hacía ganar a un auto desde noveno	resultado absurdo en una simulación	se encogió por la confiabilidad medida
La carrera duraba **71 vueltas de 72**	un plan descrito sumaba 71	`from_lap` es la primera vuelta que falta correr: entra en la cuenta
El salón de la fama de DEAP se calculaba y se **descartaba**	un test de «el mejor nunca empeora» falló	entra en el resultado
La reparación de B6.3.8 devolvía una tanda más larga que la vida del compuesto	lo encontró la suite de tests nueva	se vuelve a aplicar el control de presupuesto
El export que alimenta la interfaz quedó **un mes sin regenerar**	apareció al verificar otro arreglo	regenerado y sincronizado

Dos merecen una nota porque son la misma lección: **arreglar un error donde se lo encontró no es arreglarlo**. La referencia de pérdida de boxes se corrigió en el simulador y la versión equivocada siguió viva en el camino que alimenta la pantalla. Y la pérdida de boxes se sorteaba de una forma asumida mientras el desgaste se sorteaba de la medida, en el mismo archivo y a veinte líneas de distancia. Las dos veces el síntoma fue que **ningún número se veía mal**.

---

5. Resultados

Qué se está evaluando: probabilidad, no tiempo real

	Decisión en tiempo real	Pronóstico pre-carrera
Cuándo se decide	cuando aparece la bandera	antes de largar
Qué sabe de la neutralización	que salió, en qué vuelta, dónde está cada auto	con qué frecuencia suele salir y cuánto dura
Qué optimiza	esta carrera	el promedio sobre las carreras posibles
Es el entregable de	la Entrega 3	**éste**

De ahí sale que **una parada tomada bajo bandera y una tomada en verde no son el mismo dato**: la segunda es una decisión comparable contra un pronóstico, la primera es en buena medida un regalo.

Casos resueltos: cuatro pruebas contra carreras reales

Monza 2026 — falló, y por qué falló es el resultado más útil. Antonelli largó 19.º con duro, paró dos veces y ganó con medio; el modelo recomendaba una parada al duro. Bandera roja en la vuelta 3 y VSC en la 27-29: el **90,6%** de las paradas estratégicas fueron gratis, contra el 23% medido sobre 103 carreras.

Escenario	Plan recomendado	Lo que hizo ANT	Diferencia
Paradas a precio normal	62,77 s	81,98 s	**+19,20**
Paradas gratis	39,84 s	36,82 s	**−3,02**

El vuelco vale **22,2 segundos** y la cantidad óptima de paradas se invierte.

Madrid 2026 — acertó la forma, falló los números. Circuito nuevo, sin desgaste propio: corrió con el promedio de la temporada. Predijo una parada y la forma del plan ganador —medio corto y después duro largo—; falló el desgaste por un factor de tres y predijo que el duro correría menos del 10% de las vueltas cuando corrió el 76%.

Escenario	Modelo	Antonelli	Diferencia
supuestos previos	75,70 s	80,22 s	+4,52
+ desgaste real	32,14 s	28,19 s	**−3,95**

El signo se da vuelta corrigiendo una sola cosa. Y corriendo la búsqueda entera con los insumos reales propone `M17-H40` contra el `M14-H43` del ganador: tres vueltas. **El algoritmo nunca estuvo roto; los insumos sí.**

Elección de compuesto — falló, y es el límite más caro. El modelo pone medio **cero veces de veintidós**; la realidad lo pone entre el 55% y el 85%. El motivo es estructural: `COMPOUND_OFFSET_S` vale cero —el escalón de ritmo entre compuestos no se pudo identificar— así que el modelo elige por durabilidad y un compuesto intermedio en todo no puede ganar.

Bakú 2026 — el pronóstico pre-registrado. Se cargó la clasificación, **no** la carrera, se escribieron cuatro criterios de falsación y se commitearon. Recién entonces se miró el resultado.

Criterio	Predicho	Real	
compuesto de salida	medio 12-17, duro 4-8, blando ≤3	medio 12, duro **0**, blando **10**	error
paradas	una, ≥60%	**dos, 94%**	error
secuencia	medio → duro	**medio → blando → blando**	error
neutralización	al menos una	safety car dos veces	acierto

Uno de cuatro. Pero los tres fallos son el mismo: **35 de las 38 paradas** se hicieron bajo neutralización, y contando sólo las de bandera verde **catorce de los dieciséis clasificados no pararon ninguna vez** — que es lo que el pronóstico de una parada describía. El criterio se había

escrito sobre la cuenta cruda, y eso no se arregla a posteriori.

Tres pruebas más, ya escritas y sin resolver

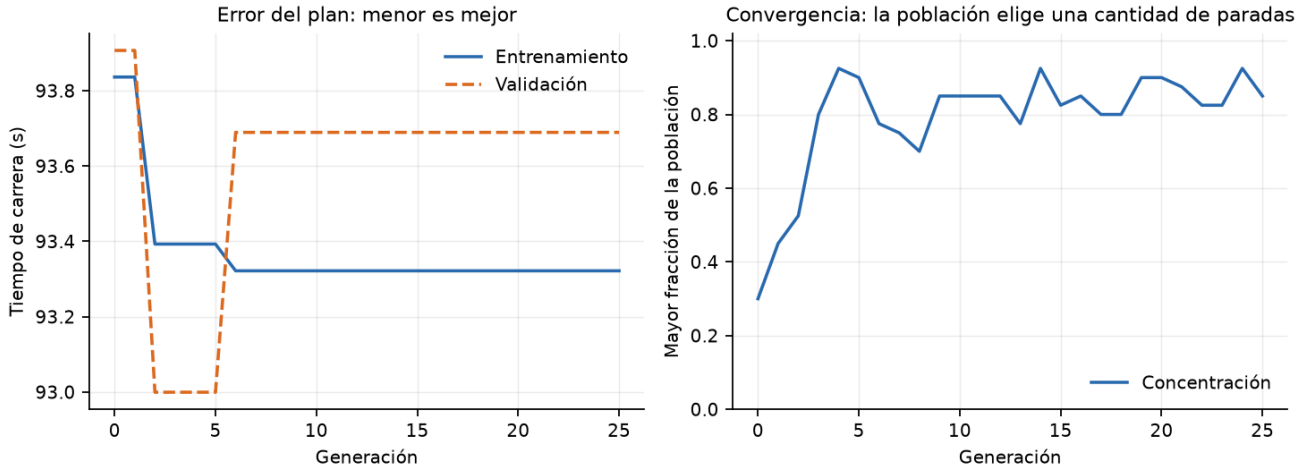
	Circuito	Qué sabe el simulador	Estado
R16	Kuala Lumpur	**nada**: cero vueltas, de ninguna temporada	corrida; sólo se cargó la clasificación
R17	Marina Bay	4.439 vueltas de 2022-2025, nada de 2026	corrida; sólo se cargó la clasificación
R18	Austin	4 carreras 2022-2025, nada de 2026	**no corrió todavía**

En las tres el historial y el modelo **se contradicen de frente**. Y hay un resultado que no depende de que corran: el modelo da **el mismo plan para Kuala Lumpur y para Austin** — `H31-S25` — porque ambos son de 56 vueltas y ninguno tiene datos de 2026. **Sin información del año en curso, el modelo no distingue circuitos: distingue distancias.**

Las métricas

Qué	Cómo	Resultado
Regresor de degradación	MAE contra predecir la mediana	0,943 contra 0,764 — **pierde** con 2026 solo; gana con todas las temporadas (0,698 contra 0,726)
Clasificador de parada	PR-AUC contra el azar	0,168 contra 0,034 = **4,99×**
Algoritmo genético	segundos contra reglas enunciables	**+10,8 a +10,9 s**
Reglamento	planes ilegales generados	**cero**
Pronóstico	criterios de falsación escritos antes	1 de 4 en Bakú
Tests automatizados	núcleo del simulador	143 tests, 1,7 s; **dos defectos encontrados**

La exactitud no se usa como métrica, y el motivo está medido: «no parar nunca» acierta el **96,6%** de las vueltas, y además de inútil es ilegal.



La curva izquierda es el análogo de una curva de pérdida: el trazo azul es la aptitud contra la que la búsqueda optimiza y baja monótonamente porque el motor es elitista; el naranja es el mismo plan sobre carreras que no intervinieron en elegirlo. Terminan en 93,32 y 93,69 s, y la brecha de 0,37 s es el optimismo que el sistema ya informaba.

---

6. Análisis de los Resultados

Las tres pruebas de *cuándo* parar las decidió una neutralización. Monza, Madrid y Bakú. El sistema acierta **que** va a haber una neutralización, con qué frecuencia y cuánto dura; no acierta **cuándo**, y el cuándo es lo que fija el plan. Ninguna cantidad de datos levanta ese techo: hay que cambiar el entregable de un plan a una política.

El algoritmo entrena de más. Con la aptitud en muestra nunca empeora, porque es elitista; medido sobre carreras que no vio, **la última generación no es la mejor en 16 de 24 corridas**. Eligiendo la generación en un juego de validación y juzgándola en otro de prueba —tres particiones, para que elegir no contamine el juicio— parar antes gana 0,387 s cuando gana, **nunca pierde**, y rinde 0,258 s de media: del orden de toda la ventaja del algoritmo sobre la mejor regla enunciable.

Por qué es difícil ganarle a una regla razonable. Desde mitad de carrera el espacio es casi unidimensional —una parada, y la única pregunta es en qué vuelta— y esa dimensión vale 0,30 s. Donde el algoritmo se gana el sueldo es **antes de largar**, decidiendo cuántas paradas.

Los dos techos, que conviene no confundir. Los de **datos** se arreglan midiendo mejor: el desgaste se aplana por supervivencia, el tráfico de rezagados queda afuera por construcción, y la asignación de neumáticos no está —lo que hace que la recomendación no sea ejecutable para el 85% de los autos—. Los **estructurales** no: que el modelo no vea el ritmo de los compuestos, que los rivales no anticipen, que se optimice de a un auto cuando los equipos tienen dos, y que el safety car pueda volver a salir sin que el modelo tenga dónde ponerlo.

Contra trabajos previos. Hay un paper de 2025 (Chaudhary et al.) que hace algo parecido con redes neuronales sobre 2020-2024 y llega a F1 = 0,81. Lo nuevo acá no es la técnica sino el encuadre: devolver una distribución en vez de una etiqueta, y evaluarse contra carreras que no

se vieron. Contra un trabajo propio anterior —un multiperceptrón de 782 registros— la diferencia es de datos: 117.545 vueltas contra 782.

---

7. Conclusiones

**Como constructor de planes el sistema funciona; como pronosticador de carreras concretas,
todavía no.** Las dos mitades están medidas y conviene no mezclarlas.

Los hallazgos, en orden de importancia:

1. **El algoritmo no estaba roto; los insumos sí.** Sale de Madrid, que es la prueba capaz de separar las dos cosas. Dice dónde está el retorno de mejorar el sistema y dónde no: afinar la búsqueda no es donde está.
2. **La incertidumbre es más grande que la señal**, y dos números lo dicen: el desvío de la tanda supera a su mediana, y el objetivo no correlaciona con nada por encima de 0,31. Por eso el sistema devuelve distribuciones y simula.
3. **El cuándo de la neutralización es un techo estructural**, y decidió tres de las cuatro pruebas.
4. **El valor del algoritmo es acertar la cantidad de paradas** —12,5 s—, no la vuelta exacta —0,30 s—. Medir lo segundo y concluir sobre lo primero fue el error de método más caro.
5. **El algoritmo entrena de más**, y parar temprano nunca pierde.
6. **El reglamento es el que manda**: la decisión real no es cuál compuesto es más rápido sino en qué tanda gastar la obligación de B6.3.8.
7. **La mitad del trabajo fue encontrar errores propios**, y el método de encontrarlos —esperar a que algo aguas abajo salga absurdo— es en sí un resultado.

Qué queda construido

El pipeline de datos sobre 106 carreras y 117.545 vueltas; el simulador Monte Carlo con todo lo que sortea medido; el algoritmo genético con dos motores comparados; una interfaz que anima una carrera sorteada; y una suite de 143 tests que encontró dos defectos el mismo día que se escribió.

Cursos de acción, y su viabilidad

Acción	Por qué	¿Se puede?
Parada temprana sobre un juego retenido	gana 0,258 s de media y nunca pierde	**Sí**, y es lo más barato de la lista
Contar despliegues de safety car, no corridas contiguas	11 despliegues contra 8 períodos en 2026	**Sí**
El **escalón de ritmo entre compuestos**	sin él, el duro sólo existe por el reglamento	Necesita telemetría o un modelo de combustible mejor
Restricción de **asignación de neumáticos**	sólo el 15% de los autos monta dos duros frescos	Sí, hay que decidir de dónde sale la asignación
Tráfico de **rezagados**	28,4% de los clasificados terminan al menos una vuelta abajo	Requiere comparar por posición y no por vuelta
Agente de **Refuerzo**	la comparación propuesta: planificar contra reaccionar	Para la Entrega 3

Lo que no se va a hacer: más términos en la aptitud esperando que deje de ser plana. El tráfico se midió, se implementó correctamente y no movió el resultado; el escalón de compuestos se midió y no se pudo transferir. Dos negativos seguidos son suficiente señal.

Referencias

Chaudhary, S. et al. (2025). *Data-driven pit stop decision support for Formula 1 using deep learning models*. Frontiers in Artificial Intelligence, vol. 8.

Fédération Internationale de l'Automobile (2026). *2026 Formula 1 Sporting Regulations — Section B*, Issue 05. Artículos B6.1.1, B6.1.2 y B6.3.8.

Fortin, F. et al. (2012). *DEAP: Evolutionary Algorithms Made Easy*. Journal of Machine Learning Research, vol. 13, pp. 2171–2175.

Harris, C. R. et al. (2020). *Array programming with NumPy*. Nature, vol. 585, pp. 357–362.

Hunter, J. D. (2007). *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering, vol. 9, n.º 3, pp. 90–95.

Ke, G. et al. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*. NIPS 2017.

Pasqualino, F.; Denoya, A.; Sánchez, C.; Sánchez, T.; Lingeri, M. (2025). *Clasificación de Compuesto para la Fórmula 1 utilizando una Red Neuronal Artificial del Tipo Multiperceptrón*. TP N.º 2, Inteligencia Artificial, UTN FRBA. https://github.com/FrancoP08/TP2-IA-2025

Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, vol. 12, pp. 2825–2830.

Pytel, P. (2026). *Inteligencia Artificial Avanzada — Programa y Planificación, 2.º cuatrimestre de 2026*. UTN FRBA, Ingeniería en Sistemas de Información.

Schäfer, P. et al. (2026). *FastF1*, versión 3.8.3. https://docs.fastf1.dev/

Código y datos

Repositorio: https://github.com/tomasanchez/boxbox

Incluye el código fuente completo, los registros de decisión de diseño y los cuadernos con el cálculo a la vista. **Los datos** de entrenamiento, validación y prueba no se versionan como archivo: se descargan de la API de FastF1 y quedan en caché local. Son las 117.545 vueltas de 2022 a 2026 descritas en «Materiales Disponibles», reconstruibles de forma reproducible.